



TECHNICAL HANDOFF DOCUMENT

The 4D Restaurant Menu Platform

A multi-tenant restaurant ordering & operations SaaS — full architecture, code map, and the open problems we need help with.

This document is written for a software developer who has never seen this codebase. By the end you should understand what the product is, how it is built (front end, back end, database, hosting), how a single order flows through the whole system in real time, and exactly which problems are still unsolved.

Project: "My Little French House" 4D Menu · internal repo name `backup_Menu` / `menu`

Prepared: 27 June 2026 · **Branch:** main · **Status:** Live in production (single restaurant), multi-tenant rollout in progress

Codebase size: Next.js 16 app · ~70 React components & lib modules · 3 vanilla JS staff panels · 104 SQL migrations · ~40 verification/seed/stress scripts

Latest work (this week): the manager floor was rebuilt as a two-tier "Table view" with incremental tile patching + event delegation to survive 300 tables (the old freeze — see Section 13, Problem 1, now largely fixed); one-call bulk "Open all / Close all"; and the admin overview gained clickable KPI detail drawers, a passwordless issues-resolve page, and per-graph date-range pickers.

Contents

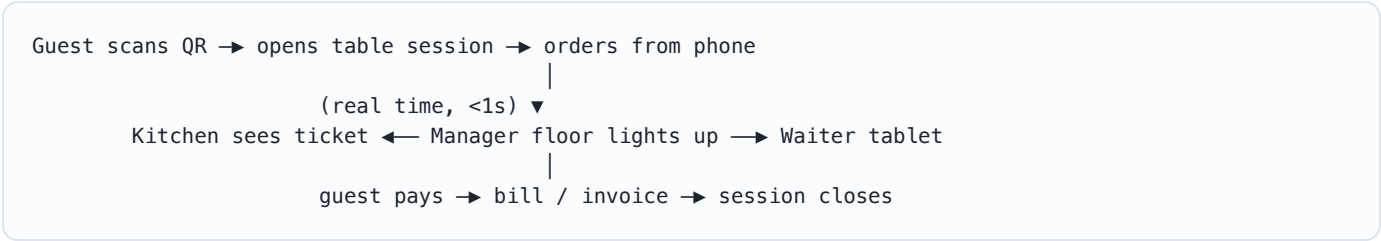
- 1 What this product is
- 2 The technology stack (front end, back end, infra)
- 3 High-level architecture (one app, six panels)
- 4 The six panels in detail
- 5 Routing & multi-tenancy (how a restaurant is resolved)
- 6 The back end: Supabase, RLS & RPCs
- 7 The data model (tables)
- 8 ★ The real-time engine — what happens when an order updates
- 9 Cost & egress design (why it is built this way)
- 10 Security model
- 11 Deployment, hosting & environments (Vercel)
- 12 How to run it locally
- 13 ⚠ OPEN PROBLEMS — what we need help solving
- 14 Full folder structure
- 15 Migration index (all 101)
- 16 Where a new developer should start

What this product is

This is a **restaurant ordering and operations platform**. A guest sits at a table, scans a QR code, and gets a rich digital menu (with optional 3D / "4D" dish models). They open a dining "session" for their table, build a cart, and place orders straight from their phone. Behind the scenes, staff run the whole floor from a set of browser panels: a **manager** panel (live floor, orders, bills), a **kitchen** display, a **waiter tablet**, an **owner** dashboard (analytics, staff, settings), and a hidden **admin** super-panel that can reach into any restaurant.

It started as a single-restaurant app for one café ("My Little French House") and is now being converted into a **multi-tenant SaaS**: one shared back end serving many restaurants, each with its own branding, menu, staff and data, isolated at the database level.

The core value loop



Key product facts

Aspect	Detail
Primary users	Restaurant guests (phone), waiters, kitchen, managers, owners, and the platform admin.
Live status	One restaurant fully live in production. Multi-tenant phases 0–5 built; restaurant #1 unchanged.
Standout feature	Per-dish 3D models ("4D menu") streamed as <code>.glb</code> files into a <code><model-viewer></code> .
Money handling	Server-authoritative pricing — the browser never decides a price; the database recomputes every order.
Real-time	Every panel updates within ~1 second of any change, using a lightweight "breadcrumb" notification system (Section 8).
Languages	Menu supports 6 languages (en/de/fr/ar/hi/ko) via JSONB translation columns.

The technology stack

Front end

Tech	Version	Role
Next.js (App Router)	16.2.6	The whole framework: server-rendered pages, client components, API route handlers, image optimization. Runs on port 4000.
React	19.2.4	UI library. Uses the React 19 compiler; strict mode on.
TypeScript	5.x (strict)	All app code is typed. Path alias <code>@/*</code> → repo root.
Tailwind CSS	4 (via PostCSS)	Utility-first styling for the React side. Global recipe in <code>app/globals.css</code> .
GSAP	3.15	Animations — intro splash and hero title only.
Recharts	3.9	Charts in the owner / admin analytics dashboards.
<model-viewer>	(CDN web component)	Renders the 3D dish <code>.glb</code> models. Loaded via a <code><Script></code> tag inside <code>components/PublicModelViewer.tsx</code> .
Vanilla JS	—	The manager / kitchen / tablet panels are hand-written HTML+CSS+JS served from <code>public/panels/</code> (see Section 4).

Back end

Tech	Role
Supabase (hosted PostgreSQL)	The single source of truth: database, Row-Level Security, Realtime (WebSockets), Storage (the 3D model files), and an anon/service-role key pair.
@supabase/supabase-js 2.106	The DB/Realtime client. Two ways it ships: bundled into the React app, and self-hosted at <code>public/vendor/supabase.js</code> for the vanilla panels (built by <code>scripts/build-vendor.mjs</code>).
Next.js Route Handlers	The server "API" lives in <code>app/api/**</code> . These use the service-role key (full DB power) and are the only place trusted business logic runs server-side besides the database functions.
PostgreSQL functions (RPCs)	The real business logic. ~80 <code>plpgsql</code> functions (order placement, pricing, sessions, bills, floor state). Most are <code>SECURITY DEFINER</code> — see Section 6.

Infrastructure & tooling

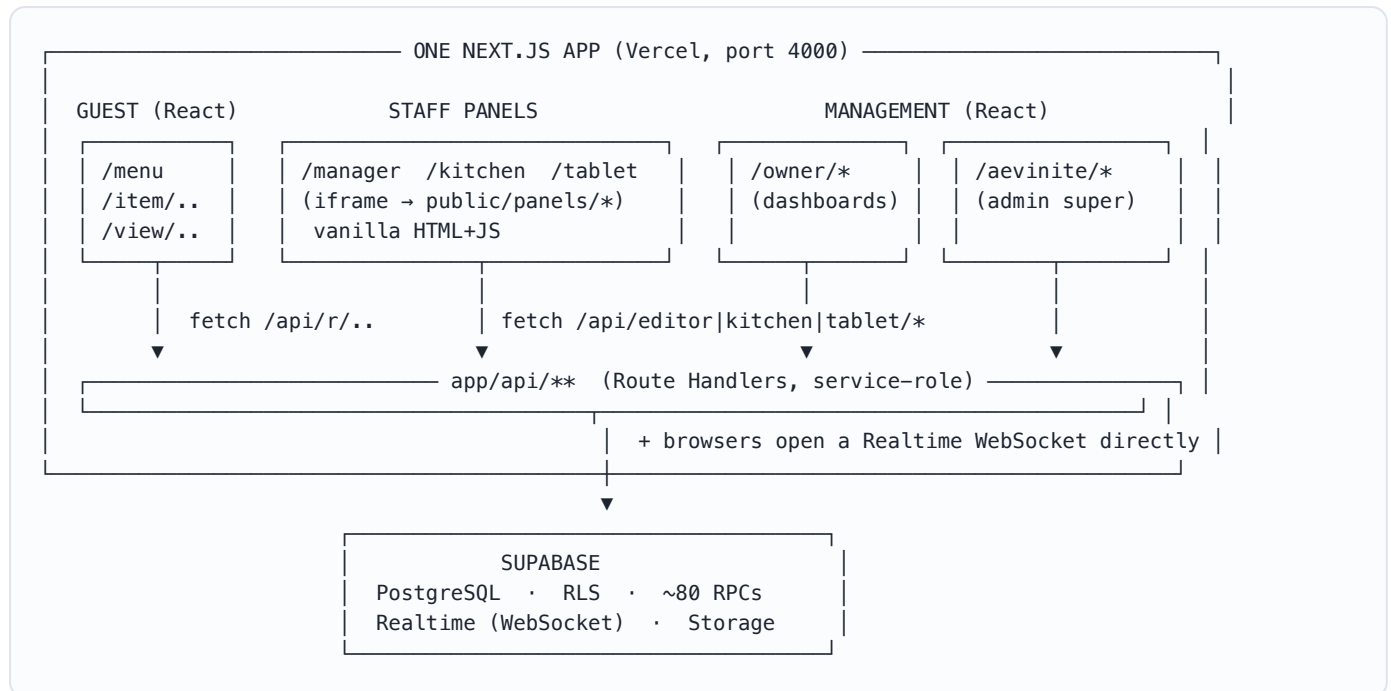
Tech	Role
Vercel	Hosting / deploy. Project <code>3-d-backup</code> . Auto-deploys <code>main</code> from GitHub. (Section 11.)
GitHub	Source repo <code>aevinite/3D_BACKUP</code> , branch <code>main</code> (protected, PR-only).
Sentry (@sentry/nextjs 10.57)	Error & performance monitoring (client, server, edge). 100% trace in dev, 10% in prod.
esbuild 0.28	Bundles the self-hosted supabase client for the vanilla panels.
Playwright 1.60	Drives the browser for the <code>scripts/verify-*.mjs</code> end-to-end checks.
ESLint 9 + eslint-config-next	Linting. <code>npm run lint</code> .

npm scripts (package.json)

```
dev      next dev -p 4000      # local dev server
build    next build           # production build (prebuild checks 3D cache headers)
start    next start -p 4000   # run the production build
lint     eslint
build:vendor node scripts/build-vendor.mjs # rebundle supabase-js for the vanilla panels
test:money node --test tests/money.test.mjs # pure money-math unit tests
test:totals node --test tests/order-totals.e2e.mjs # client price == server price, to the cent
cache-models node scripts/set-glb-cache.mjs # ensure GLB files have year-long cache headers
```

High-level architecture

Everything is **one Next.js application** on a single port (4000). It used to be four separate servers; they were merged. The different "panels" are just routes inside the one app. Some panels are modern React; three operational panels (manager/kitchen/tablet) are older vanilla-JS pages embedded full-screen via an `<iframe>`, with their data served by Next API routes.



One app, many panels, one database. The browser talks to the DB two ways: through our API routes (reads/writes) and directly over a Realtime WebSocket (to hear about changes).

The three "layers" of logic

1. **Browser (UI):** never trusted with money or permissions. It only displays state and sends intents ("place this cart", "mark item served").
2. **API routes (`app/api/**`):** hold the service-role key, gate by role/cookie, and call the database.
3. **Database functions (RPCs):** the real rules. Pricing, session validity, bill numbering, floor state — all enforced in PostgreSQL so they can't be bypassed.

The six panels in detail

Panel	Route	Tech	Who	What it does
Guest menu	<code>/menu</code> , <code>/item/[slug]</code> , <code>/view/[folder]</code>	React	Customers	Browse menu, 3D dish viewer, open table session, cart, place order, track status, leave feedback/bill.
Manager (was "editor")	<code>/manager</code>	Vanilla JS in iframe	Floor managers	"Table view" — a two-tier floor (slim summary grid → detail-on-tap) built to scale to ~300 tables; orders, per-order discount, KOT chips, shift table, one-call bulk Open all / Close all, bills, dishes/categories/tags editor, customers, logs.
Kitchen (KDS)	<code>/kitchen</code>	Vanilla JS in iframe	Cooks	New → Cooking → Ready ticket board, "86" (sold-out) board, chime on new order.
Tablet	<code>/tablet</code>	Vanilla JS in iframe	Waiters	Floor tiles + "take order" for a table (captain mode), approve join requests.
Owner	<code>/owner/*</code>	React	Restaurant owner	Sales/analytics, orders, customers, staff management, manager-power grants, menu, marketing, settings, issues, inventory (planned).
Admin (super-panel)	<code>/aevinite/*</code>	React	Platform admin	Reach ANY restaurant invisibly: restaurants, owners, users, features/entitlements, billing, floor, analytics, logs, health, settings, "act-as". Overview has clickable KPI detail drawers (zero extra DB reads), a passwordless issues-resolve page (<code>/aevinite/issues</code>), and per-graph date-range pickers (today/yesterday/7d/30d/all).

Build status — what is actually implemented (don't assume every sub-page is live)

The guest menu and the manager / kitchen / tablet panels are **fully working in production**. The newer React management panels are partly scaffolded — several sub-pages are intentional `ComingSoon` placeholders:

- **Owner** — live: `home/overview`, `report` , `staff` , `issues` . Placeholder (`ComingSoon`): `customers` , `orders` , `sales` , `menu` , `marketing` , `online` , `settings` , `inventory` .
- **Admin** (`/aevinite`) — live: `overview` , `restaurants` , `users` , `features` , `floor` , `logs` , `issues` , `settings` . Placeholder: `analytics` , `owners` , `billing` , `health` (the overview already shows real charts + KPI drawers that cover much of these).

Placeholders render a tidy "coming soon" card via `components/ComingSoon.tsx` — so an empty page is by design, not a bug.

Why the manager/kitchen/tablet panels are an `<iframe>`

They are the original pre-rewrite vanilla pages (`public/panels/editor` , `/kitchen` , `/tablet`). The Next routes (`app/manager/page.tsx` etc.) are 15-line wrappers that just embed the static page full-screen. The internal name "editor" survives in the file paths and API routes (`/api/editor/*`) even though the user-facing route is `/manager` — renaming was judged not worth the risk. The single biggest vanilla file is `public/panels/editor/app.js` (~4,360 lines) and it holds most of the floor logic.

Shared panel scripts (`public/panels/`)

File	Purpose
<code>realtime.js</code>	Shared real-time helper for all three vanilla panels: one WebSocket, breadcrumb listener, debounced refetch, reconnect/idle handling. (Section 8.)
<code>backstack.js</code>	Hardware back-button manager — every popup/drawer registers so the phone back button peels one layer instead of quitting.
<code>maint.js</code>	Maintenance-mode banner/gate.
<code>theme.js</code>	Panel theming.
<code>vendor/supabase.js</code>	Self-hosted Supabase client (built by esbuild) so panels never depend on a public CDN that a restaurant's Wi-Fi might block.

Routing & multi-tenancy

Tenant resolution (which restaurant?) is centralized in `lib/tenant.ts` — one resolver, written so the source of the slug can change later without rewriting routes.

- **Today (path-based):** the QR code encodes a slug + table, e.g. `/r/<restaurant-slug>/menu` and `/r/<restaurant-slug>/item/<dish>`. `getRestaurantBySlug(slug)` reads the public `restaurants` table (anon SELECT allowed) and caches the result per process.
- **Legacy single restaurant:** bare `/menu` still resolves to restaurant #1 (`00000000-0000-0000-0000-000000000001`, slug `french-house`) so the original live site never broke.
- **Later (subdomain / custom domain):** the same resolver can read the slug from the host header — a DNS/config flip, not a rewrite.

Route map

```

/                → redirect to /menu
/menu           → guest menu (restaurant #1)
/r/[restaurant]/menu → guest menu for a specific tenant
/item/[slug]    → dish detail           /r/[restaurant]/item/[slug]
/view/[folder]  → full-screen 3D viewer
/manager        → manager panel (iframe → /panels/editor)
/editor         → redirects to /manager (back-compat)
/kitchen        → kitchen display (iframe → /panels/kitchen)
/tablet         → waiter tablet (iframe → /panels/tablet)
/owner/*        → owner dashboards (customers, orders, sales, report,
                  staff, settings, menu, marketing, online, issues, inventory)
/aevertime/*    → admin super-panel (restaurants, owners, users, features,
                  billing, floor, analytics, issues, logs, health, settings)
/login, /staff-login → auth pages

```

API surface (`app/api/**`)

Route group	What it serves
<code>/api/r/[restaurant]/menu-data</code>	Cached guest menu payload, scoped to one restaurant.
<code>/api/editor/[...path]</code>	Catch-all backing the manager panel: orders, calls, sessions, dishes, etc. (e.g. <code>/orders?table=3</code>).
<code>/api/kitchen/[...path]</code> , <code>/api/tablet/[...path]</code>	Catch-alls backing the kitchen & tablet panels.
<code>/api/owner/*</code>	overview, analytics, staff, manager-permissions, issues.
<code>/api/admin/*</code>	overview, restaurants(+features), users, settings, floor, maintenance, oplog, custlog, act-as .
<code>/api/aggregators/webhook/[source]</code>	Inbound Zomato/Swiggy/takeaway order webhooks.
<code>/api/rt-config</code>	Hands the vanilla panels the PUBLIC supabase url + anon key (never the service-role key) so they can open a WebSocket.
<code>/api/staff-login</code> , <code>/api/panel-login</code> , <code>/api/panel-profile</code> , <code>/api/maintenance</code>	Auth & misc.

The back end: Supabase, RLS & RPCs

The golden rule: the client is never trusted

Two design decisions make this safe:

1. **Row-Level Security, locked by default.** Most tables have RLS *enabled but with no public policies*. That means the anon (browser) key cannot directly read or write them at all. The only way in is through database functions.
2. **SECURITY DEFINER functions.** Guest-facing operations are PostgreSQL functions that run with elevated rights and contain all the validation. The browser calls `lfh_place_order(token, items, allergies)` ; it cannot insert into `orders` directly. Every staff-only function is additionally locked with `REVOKE ... FROM PUBLIC, anon, authenticated` and `GRANT ... TO service_role` .

Server-authoritative pricing (the most important pattern)

The browser sends only item id + qty + chosen options + removals + note. The function `lfh_price_order()` re-reads every dish from the database, rejects sold-out items, recomputes the price and tax, and applies the same "nice rounding" the UI shows. A guest (or attacker) literally cannot change a price. There's even a test (`npm run test:totals`) proving the client's displayed price matches the server's to the cent.

The most important RPCs

Function	Who can call	Job
<code>lfh_price_order(items)</code>	read-only	Recompute subtotal/tax/total from the DB. Single source of price truth.
<code>lfh_place_order(token, items, allergies)</code>	guest (anon)	Validate the guest's session token, price the order, insert <code>orders + order_items</code> .
<code>lfh_staff_place_order(table, items, ...)</code>	service_role	Waiter places an order for a table (captain mode).
<code>get_order_status(order_id)</code>	guest (anon)	Guest polls only their own order by exact id; returns 4 safe columns.
<code>lfh_open_session / join / approve / leave</code>	guest	The dining-session lifecycle (open a table tab, add members, approve joiners).
<code>lfh_floor_state()</code> / <code>lfh_floor_bundle()</code>	service_role	"The brain": authoritative status of every table in one call (free/seated/new/preparing/served, dues, calls). Backs the manager floor.
<code>lfh_kitchen_tickets()</code>	service_role	The kitchen's slice — live orders with per-item statuses.
<code>lfh_assign_kot / assign_bill / generate_invoice / void_invoice</code>	service_role	Daily KOT numbers, per-session bill numbers, permanent invoice sequence + voiding.
<code>lfh_staff_shift_table(session, to)</code>	service_role	Atomically move a party from one table to another.
<code>lfh_auto_close_idle_sessions()</code>	service_role	Presence-aware safety net: closes abandoned table tabs (no blocking order, idle > N min).
<code>lfh_rt_emit()</code> (trigger)	internal	The real-time "breadcrumb" emitter — see Section 8.
<code>lfh_owner_overview / revenue / hourly / dish_breakdown</code>	service_role	Pre-shaped analytics for the owner/admin dashboards (indexed, not full scans).

Gotcha for any new function

New PostgreSQL functions are **PUBLIC-executable by default**. Every new staff-only function must be locked (`REVOKE` from public/anon/authenticated, `GRANT` to service_role) — this was caught once in production (migration 038).

The data model

Every tenant-scoped table carries a `restaurant_id` (added additively in migration 078, defaulting to restaurant #1). Uniqueness, counters and indexes were all rescoped per-restaurant in 079–082.

Catalog & config

Table	Purpose & key columns
<code>restaurants</code>	The tenant root. <code>id</code> , <code>slug</code> (unique), <code>name</code> , <code>active</code> , <code>subdomain</code> , <code>custom_domain</code> , <code>logo_text</code> , <code>hero_title</code> , <code>tagline</code> , <code>accent_color</code> .
<code>menu_items</code>	Dishes. <code>restaurant_id</code> , <code>slug</code> , <code>title</code> , <code>price</code> (text), <code>category</code> , <code>image</code> , <code>tags[]</code> (e.g. <code>veg</code> , <code>sold-out</code>), <code>options</code> (jsonb), <code>allergens[]</code> , <code>is4d</code> , <code>model*_url</code> .
<code>categories</code> / <code>filters</code>	Menu category tiles & filter chips. Multilingual <code>name</code> (jsonb), <code>icon</code> , <code>color</code> , <code>sort_order</code> , <code>active</code> . Composite PK (<code>restaurant_id</code> , <code>slug</code>).
<code>settings</code>	One row per restaurant. Feature toggles + geofence (<code>geo_lat/lng/radius_m</code>), <code>sessions_enabled</code> , <code>require_otp</code> , <code>table_count</code> , <code>platform_in_bills</code> , etc.

Dining sessions & guests

Table	Purpose & key columns
<code>sessions</code>	An open "tab" for a table. <code>table_number</code> , <code>status</code> (open/closed/pending), <code>opened_by</code> , <code>last_activity_at</code> (idle tracking), <code>bill_no</code> , <code>invoice_no</code> , <code>invoice_voided</code> , <code>cart</code> (jsonb, shared). Unique: one open session per (restaurant, table).
<code>session_members</code>	The phones in a session. <code>phone</code> , <code>phone_verified</code> , <code>name</code> , <code>token</code> (access pass), <code>role</code> (owner/guest), <code>approved</code> , <code>location_ok</code> , <code>removed</code> .
<code>customers</code>	Returning-guest recognition. PK (<code>restaurant_id</code> , <code>phone</code>); <code>name</code> , <code>blocked</code> , <code>first/last_seen</code> .
<code>requests</code>	Queue of open/join/access decisions awaiting staff or host approval.
<code>blocklist</code> / ban system	Bans by phone / table / member (migration 077).
<code>otp_codes</code> / <code>verification_codes</code>	One-time codes for optional phone verification.

Orders, money & ops

Table	Purpose & key columns
<code>orders</code>	A charge at a table. <code>table_number</code> , <code>session_id</code> , <code>member_id</code> , <code>items</code> (jsonb), <code>subtotal</code> , <code>tax</code> , <code>total</code> , <code>discount(+note)</code> , <code>allergies[]</code> , <code>status</code> (received/preparing/served/cancelled), <code>payment_status</code> , <code>archived</code> , <code>kot_no</code> , <code>created_at</code> .
<code>order_items</code>	Per-item rows so the kitchen can track each line. <code>title</code> , <code>qty</code> , <code>unit_price</code> , <code>options</code> , <code>removed[]</code> , <code>note</code> , <code>status</code> , <code>served_at</code> .
<code>waiter_calls</code>	"Call waiter" requests, de-duped, resolved by staff.
<code>aggregator_orders</code>	Zomato/Swiggy/takeaway. <code>source</code> , <code>external_id</code> , <code>payload</code> , <code>items</code> , <code>total</code> , <code>status</code> (new→accepted→...→handed_over), <code>status_history</code> , <code>kot_no</code> .
<code>payments</code>	Transactions per session.
<code>daily_counters</code> / <code>seq_counters</code>	KOT & bill reset daily (05:00 IST business day); invoice numbers are forever-sequential. PKs include <code>restaurant_id</code> .
<code>feedback</code> / <code>reviews</code>	Session feedback & one rating per device per dish.

Staff, audit & real-time

Table	Purpose & key columns
<code>staff_users</code>	Staff accounts. <code>username</code> , <code>password_hash</code> , <code>role</code> (manager/kitchen/tablet), <code>name</code> , <code>active</code> .
<code>restaurant_owners</code>	Which owner account owns which restaurant(s).
<code>staff_actions</code>	Audit log; also drives the admin live activity feed.
<code>issues</code>	In-app bug / feedback tracker (migration 094).
<code>realtime_events</code>	The breadcrumb table. Tiny rows: <code>topic</code> ('ops'/'menu'/'table:N'), <code>kind</code> , <code>entity_id</code> , <code>table_number</code> , <code>restaurant_id</code> , <code>created_at</code> . This is the heart of the live-update system — Section 8.

★ The real-time engine: what happens when an order updates

You asked specifically: *"when an order updates, what auto-refreshes — the whole path."* Here it is, end to end. This is also the part most relevant to the performance problems in Section 13.

The idea in one sentence

Instead of every panel re-reading the whole floor every second (expensive), the database drops a tiny **"breadcrumb"** note whenever something changes; panels listen for that note over a WebSocket and then refetch **only the table that changed**.

The full path of a single order change

- ① A waiter taps "Served" on table 3
 - └ POST /api/editor/... → UPDATE order_items SET status='served' WHERE id=...
- ② A database TRIGGER fires automatically: lfh_rt_emit()
 - └ INSERT INTO realtime_events (topic='ops', kind='order_item',
table_number='3', restaurant_id=..., created_at=now())
(a ~100-byte "breadcrumb" — NOT the order data itself)
- ③ Supabase Realtime broadcasts that INSERT over a WebSocket,
filtered by topic. Every open panel is subscribed to channel rt:ops .
- ④ In the browser (public/panels/realtime.js or lib/useRealtime.ts):
 - receives the breadcrumb
 - checks restaurant_id matches this panel (ignore other tenants)
 - during a ~200ms debounce window, COLLECTS the set of changed
table_numbers → {3} (a burst of Preparing→Ready→Served = ONE refetch)
- ⑤ The manager panel's handler decides:
 - all events name a table? → pollTables(['3']) ← TARGETED
 - something unscopable (platform order / wake / reconnect)? → full reload
- ⑥ pollTables(['3']) fetches ONLY table 3's slice:
 - GET /api/editor/orders?table=3
 - GET /api/editor/calls?table=3
 - GET /api/editor/sessions?table=3
 - then MERGES: drop table 3's old rows, splice the fresh ones in,
dedupe by row id, leave every other table untouched.
- ⑦ reconcileBoard() → same redraw/chime/badge logic as a full poll.
Table 3's tile updates on every screen in <1 second. Nothing else repaints.

Who listens to what

Panel	Listener	On an "ops" breadcrumb
Manager	<code>realtime.js</code>	Targeted — <code>pollTables([changed])</code> , full reload only as fallback.
Kitchen / Tablet	<code>realtime.js</code>	Still do a full refetch on any ops event (not yet converted to targeted).
Guest menu	<code>lib/useRealtime.ts</code>	Only subscribes to the <code>menu</code> topic (menu/price changes) — guests never receive the order "firehose".

The safety nets baked into the listener

- **60-second backup poll.** If the WebSocket silently dies, a slow full refresh still corrects the screen — a missed breadcrumb can never leave a permanently-wrong floor.
- **Force-reconnect on wake.** On tab focus / visibility / `pageshow` / network-back, the socket is rebuilt (a backgrounded phone often kills it) and a full refetch runs.
- **Idle disconnect (120s hidden).** A tab left hidden drops its channels so forgotten tabs stop holding a live connection (this fixed "~41 phantom realtime users"). It reconnects instantly when shown.
- **worker:true.** The WebSocket heartbeat runs in a Web Worker so a throttled background tab doesn't quietly drop the connection.
- **Latest-wins guard.** Each refetch carries a sequence number; a slow older response is discarded so it can't overwrite newer data (the app's classic "stale refresh race").
- **Redraw fingerprint (kitchen/tablet).** Panels only repaint when a fingerprint of the data changes, preventing flicker on identical refetches.

The breadcrumb has sharp edges (where bugs hide)

- **Every rendered column must be in the trigger's watch list.** If a panel shows a column the trigger doesn't watch, edits to it produce no breadcrumb → it only updates on the 60s poll. This bit us with invoice-void columns (fixed in migration 096).
- **Table shift needs two breadcrumbs.** Moving a party A→B must emit for *both* tables or the old tile stays wrong for 60s (also fixed in 096).
- **Merge must dedupe by row id, not table number** (a shift changes a row's table number) — otherwise a tile can duplicate.

Cost & egress design

The single biggest operational fear is database **egress** (bytes read out of Supabase) and connection count, because this is billed and quota-limited. A real incident: egress hit the quota and **~96.6% of it was whole-board reads** — panels re-reading entire tables on every live tick.

The rules every query follows now

- **Scoped:** always `.eq("restaurant_id", rid)`, an explicit column list (never `select *` on a hot path), and a `.limit()`.
- **Targeted refetch:** on a live event, refetch only the changed table (Section 8 step ⑥), not the whole floor.
- **Indexed:** every filtered/sorted column is indexed — especially `(restaurant_id, created_at)` on append-heavy tables (migrations 095, 098). Owner analytics RPCs were full-scanning `orders` and taking ~147s under load before indexing.
- **One pooled connection;** realtime channels are keyed per restaurant and dropped on hidden/idle.
- **No poll faster than the 60s backstop.**

The targeted-refetch change cut order-tick egress by roughly 92% on the manager panel. Kitchen and tablet still do full refetches and are the next candidates.

Security model

- **Database is the wall.** RLS enabled, no public policies, all access via locked `SECURITY DEFINER` RPCs (Section 6). The anon key is harmless on its own.
- **Service-role key is server-only.** It lives in env vars, is used only inside `app/api/**` via `lib/supabaseAdmin.ts`, and is never sent to the browser. `/api/rt-config` deliberately exposes only the *public* url + anon key.
- **Admin gate.** Historically only `/admin` was cookie-gated; staff panels were intentionally open during single-restaurant operation and must be re-locked before broad public hosting. Admin is a separate login from owner; the admin can "act-as" any restaurant invisibly.
- **Roles:** admin > owner > manager > kitchen/tablet. No role gets blanket access; capabilities are granted deliberately, and owners can grant/revoke specific "manager powers".
- **Secrets discipline:** secrets never appear in chat/logs; `.env.local` is gitignored; the Sentry DSN is public (safe).

Deployment, hosting & environments

How it ships

- **Host:** Vercel project `3-d-backup`. Pushing to GitHub `main` (repo `aevinite/3D_BACKUP`) auto-deploys. `main` is protected — changes land via PR.
- **Build:** `vercel.json` → `npm run build` → `.next/`. A `prebuild` step checks the 3D model cache headers (safe to fail). Sentry uploads source maps in CI when its token is present.
- **One repo, one deploy** ships the guest menu + all panels + admin together. The old separate editor repo is retired.

Environments

Two Supabase projects: **production** and a **dev/sandbox** (separate `SUPABASE_DEV_*` keys) used for seeding demo data and stress tests so production is never touched.

Environment variables (names only — values are secret)

Variable	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Public Supabase project URL (shipped to browser).
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Public anon key (shipped to browser; harmless thanks to RLS).
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Full-power server key — server only, never exposed.
<code>SUPABASE_ACCESS_TOKEN</code>	Management-API token used to run migrations (DDL).
<code>ADMIN_PASSWORD</code>	Admin gate password (also set in Vercel env for prod).
<code>SESSION_SECRET</code>	Session/cookie signing secret.
<code>VERCEL_TOKEN</code>	Deploy/health API token.
<code>SENTRY_AUTH_TOKEN</code>	Source-map upload in CI.
<code>SUPABASE_DEV_*</code> (url, anon, service-role, project-ref, db-password, access-token)	The dev/sandbox database — kept strictly separate from prod.

How to run it locally

```
# 1. install
npm install

# 2. create .env.local with the keys listed in Section 11
#   (NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY,
#   SUPABASE_SERVICE_ROLE_KEY, ADMIN_PASSWORD, ...)

# 3. run the dev server (everything, port 4000)
npm run dev

#   → http://localhost:4000/menu      guest
#   → http://localhost:4000/manager  manager panel
#   → http://localhost:4000/kitchen  kitchen
#   → http://localhost:4000/tablet   waiter tablet
#   → http://localhost:4000/owner    owner dashboard
#   → http://localhost:4000/aevinite admin

# helper scripts
npm run build:vendor    # after upgrading supabase-js
node scripts/seed-supabase.mjs  # run all migrations + seed catalog (CAUTION: overwrites)
node scripts/stress.mjs        # simulate floor load (sandbox)
node scripts/verify-cache.mjs  # Playwright happy-path 3D check
```

Cache-busting note: `public/panels/editor/app.js` loads with no version query, so a browser can keep a stale copy inside the iframe after an edit. Hard-reload / disable cache when testing panel JS changes.

Convenience launchers exist for macOS (`START-ALL.command` , `run.command`) and Windows (`.bat`), which kill port 4000, install if needed, start the dev server, and open a browser.

⚠️ Open problems — what we need help solving

These are **still happening** or only partially mitigated. For each: the symptom, how to reproduce, what we already tried, and what is still failing. The architecture above is context; **this section is the actual request**.

Problem 1 — Panels jam / freeze under load largely fixed — verify at scale

Symptom (was)	When many tables and orders were active at once, the operational panels (especially manager) became sluggish or appeared to freeze: clicks lagged, the floor stuttered, the side panel "jammed".
Root cause (now confirmed)	It was client render cost, not the database : the ~4,360-line <code>public/panels/editor/app.js</code> rebuilt the <i>entire</i> floor DOM on every update, so at ~300 tables each realtime tick re-rendered hundreds of tiles on the main thread.
Fixed by	(1) Incremental tile patching + event delegation — only the changed tiles are mutated, not the whole grid (PR #62). (2) A two-tier "Table view" : a slim summary grid (<code>lfh_table_view_summary</code> , migration 101) with detail-on-tap, so 300 tables render cheaply (PR #57/#59). (3) One-call bulk Open all / Close all with optimistic tiles (migrations 102/103, PR #60/#61). Plus the earlier work: targeted per-table refetch (~92% egress cut), one-round-trip floor read (<code>lfh_floor_bundle</code> , migration 100), coalesced realtime bursts, idle disconnect.
Still to verify	Re-run <code>scripts/stress-max.mjs</code> at a full 300 tables and confirm the manager panel stays responsive end-to-end (the fixes are in; a fresh large-scale soak is the remaining confidence check). Kitchen/tablet are lighter but not yet on the incremental-patch path.

Problem 2 — Database egress & cost mitigated, not closed

Symptom	Egress spiked to the plan's quota; ~96.6% was whole-board PostgREST reads.
Reproduce	Before the fix: any live tick re-read entire tables. Watch the Network tab on kitchen/tablet today — they still full-refetch on every ops event.
Tried	Scoped queries (<code>restaurant_id</code> + column list + limit), targeted refetch on manager, indexes (095/098), pooled connection, idle channel drop.
Still failing	Kitchen and tablet panels are NOT yet converted to targeted refetch — they remain whole-board readers. Need the same <code>?table=N</code> merge pattern there, plus a standing audit that no new feature reintroduces a whole-table read.

Problem 3 — Real-time sync glitches

recurring class

Symptom	A change occasionally doesn't appear instantly on another panel: stale value for ~1s then corrects, flicker, a missed update that only fixes on the 60s poll, or (historically) a duplicated tile.
Reproduce	Shift a table A→B; void an invoice; edit a column a panel renders but the trigger doesn't watch — watch for a tile that's wrong until the 60s backup poll.
Tried	Breadcrumb watch-list fixes (096), dual breadcrumbs on shift, dedupe-by-id merge, latest-wins sequence guard, redraw fingerprint, force-reconnect on wake.
Still failing	It's a whole class of bug, not one: any new rendered column or new cross-table action can silently break the breadcrumb contract. We want a more robust pattern (or a check) so "rendered column not in the trigger watch-list" can't happen again.

Problem 4 — Other glitches to be aware of

misc

- **Stale iframe JS:** `app.js` has no cache-busting query → a `/manager` reload can run stale panel code (looks like "nothing got fixed").
- **Multi-tenant leakage:** recurring bug where restaurant #1's branding / intro splash leaks onto other tenants — every restaurant must be visually distinct.
- **Connection budget:** backgrounded/forgotten tabs holding phantom realtime connections (largely fixed by idle-disconnect, worth re-verifying at scale).
- **Redraw guard fragility:** the kitchen/tablet repaint fingerprint must include any new field or edits silently won't refresh.

What good help looks like

A developer who reads this should be able to: (1) re-run `stress-max.mjs` at 300 tables to confirm Problem 1's fix holds, and extend the incremental tile-patch path to kitchen/tablet; (2) port the targeted `?table=N` refetch to kitchen/tablet for Problem 2; (3) propose a sturdier realtime contract so Problem 3's class can't recur. Start in `public/panels/editor/app.js` (`pollTables`, `reconcileBoard`, the new incremental tile patcher), `public/panels/realtime.js`, and the `lfh_rt_emit` trigger (migrations 057–096).

Full folder structure

Generated from the live repo (excludes `node_modules`, `.next`, `.git`, build caches). The large `reference/code-snapshot/` subtree — a read-only frozen copy of the old single-restaurant app kept for comparison — is collapsed to one line to keep this readable.

```

backup_Menu/
├─ app/                                # Next.js App Router – pages + API
│  ├─ page.tsx                        # → redirect to /menu
│  ├─ layout.tsx  globals.css  not-found.tsx  favicon.ico
│  ├─ menu/page.tsx                  # GUEST menu
│  ├─ item/[slug]/                   # dish detail (page.tsx + ItemClient.tsx)
│  ├─ view/[folder]/                 # 3D viewer (page.tsx + ViewerClient.tsx)
│  ├─ r/[restaurant]/                # tenant-scoped guest routes
│  │  ├─ menu/page.tsx
│  │  └─ item/[slug]/page.tsx
│  ├─ manager/ (page.tsx, layout.tsx) # iframe → /panels/editor
│  ├─ editor/page.tsx                # redirects to /manager
│  ├─ kitchen/ (page.tsx, layout.tsx)
│  ├─ tablet/ (page.tsx, layout.tsx)
│  ├─ owner/                          # OWNER dashboards
│  │  ├─ page.tsx  layout.tsx
│  │  ├─ customers/ orders/ sales/ report/ staff/ settings/
│  │  └─ menu/ marketing/ online/ issues/ inventory/
│  ├─ aevinite/                      # ADMIN super-panel
│  │  ├─ page.tsx  layout.tsx
│  │  ├─ restaurants/ owners/ users/ features/ billing/
│  │  └─ floor/ analytics/ logs/ health/ settings/
│  ├─ login/ (page.tsx, LoginForm.tsx)  staff-login/page.tsx
│  └─ api/                            # SERVER route handlers (service-role)
│     ├─ r/[restaurant]/menu-data/route.ts
│     ├─ editor/[...path]/route.ts     # backs the manager panel
│     ├─ kitchen/[...path]/route.ts  tablet/[...path]/route.ts
│     ├─ owner/ (analytics, overview, staff, issues, manager-permissions)
│     ├─ admin/ (overview, restaurants(+features), users, settings,
│     │         floor, maintenance, oplog, custlog, act-as)
│     ├─ aggregators/webhook/[source]/route.ts
│     ├─ rt-config/route.ts            # public url+anon key for panels
│     ├─ staff-login/ staff-logout/ panel-login/ panel-logout/ panel-profile/
│     └─ maintenance/route.ts
├─ components/                        # React components (guest + admin + owner)
│  ├─ AppShell GuestChrome Header HeroTitle IntroSplash Particles
│  ├─ MenuView FoodCard VegIcon NavPicker (menu UI)
│  ├─ CartPanel MiniCart OrderConfirmModal OrderTracker (ordering)
│  ├─ SessionGate SessionOwner SessionCartSync SessionStatusWidget
│  │  SessionTableBill BanGate (dining sessions)
│  ├─ PublicModelViewer ModelToastHost (3D)
│  ├─ ChefCallButton ChefPopup StarRating ToastHost Maintenance
│  ├─ BackQuitDialog PointerCaptureGuard InfinityLoader ComingSoon
│  ├─ RealtimeProvider
│  ├─ admin/ (AdminShell, ReorderableNav, shared.tsx)
│  └─ owner/ (OwnerShell, Charts.tsx)
├─ lib/                              # shared TypeScript modules
│  ├─ supabase.ts (anon client)  supabaseAdmin.ts (service role)
│  ├─ tenant.ts (tenant resolver)  restaurant-context.tsx
│  ├─ menu.ts menuDataServer.ts  features.ts (useFeatures)
│  ├─ useRealtime.ts liveBoard.ts (realtime, React side)
│  ├─ modelLoader.ts modelWatchlist.ts (3D singleton + toasts)
│  ├─ backStack.ts                (hardware back button)
│  ├─ session.ts sessionClose.ts autoSettle.ts table.ts tableConnection.ts
│  ├─ staffAuth.ts userAuth.ts panelGate.ts panelScope.ts ownerScope.ts
│  ├─ managerPin.ts oplog.ts orderStatus.ts businessDay.ts
│  ├─ i18n.ts format.ts money.mjs allergens.ts aggregators.ts
│  └─ device.ts guestDevice.ts guestName.ts

```


- └─ public/
 - └─ panels/ # the VANILLA staff panels
 - └─ editor/ (index.html, app.js ~4360 lines, style.css) # = manager
 - └─ kitchen/ (index.html, app.js, style.css)
 - └─ tablet/ (index.html, app.js, style.css)
 - └─ realtime.js backstack.js maint.js theme.js
 - └─ vendor/supabase.js # self-hosted supabase client
 - └─ content/ (menu.json, items/*/config.json)
 - └─ models/ (croissant .glb 3D files)
 - └─ mockups/ (design mockups)
 - └─ *.svg lfh-logo.png
- └─ supabase/migrations/ # 104 SQL migrations (001 ... 103) – Section 15
- └─ scripts/ # ~40 node scripts
 - └─ seed-supabase.mjs seed-demo-* seed-today.mjs generate-cafe-menu.mjs
 - └─ apply-migration*.mjs run-migration.mjs build-vendor.mjs set-glb-cache.mjs
 - └─ stress.mjs stress-max.mjs stress-realtime.mjs measure-load.mjs vercel-load.mjs
 - └─ verify-*.mjs (cache, order, realtime, session-ux, tablet-parity, ...)
 - └─ LOCAL-DB-NOTES.md local-db-validate.mjs cleanup-stale-orders.mjs
- └─ docs/ # plans, audits, SaaS architecture, playbooks
 - └─ SAAS-ARCHITECTURE-PLAN.html SAAS-EFFICIENCY-PLAYBOOK.md
 - └─ BUSINESS-LOGIC-AUDIT.md SECURITY-AUDIT-2026-06-26.md HANDOVER-*.md
 - └─ superpowers/ (plans/ specs/)
- └─ tests/ (money.test.mjs, order-totals.e2e.mjs)
- └─ reference/ (DATABASE.md, README.md, code-snapshot/ ... FROZEN read-only copy)
- └─ next.config.ts tsconfig.json postcss.config.mjs eslint.config.mjs
- └─ vercel.json .vercelignore global.d.ts
- └─ instrumentation.ts instrumentation-client.ts sentry.{server,edge}.config.ts
- └─ package.json README.md CLAUDE.md (project rules)
- └─ START-ALL.command run.command *.bat (launchers)

Migration index (all 104)

Migrations are run in order by `scripts/seed-supabase.mjs` (or applied one-by-one via the Management API). Every change after 078 is additive and tenant-aware. Reading these top-to-bottom is the fastest way to understand how the data model and rules evolved.

001 menu_items	035 feature_flags	069 resolve_open_requests_on_open
002 categories_and_filters	036 kot_bill_staff_orders	070 auto_table_action
003 settings	037 billing_feedback_backend_stubs	071 platform_orders
004 service_mode	038 lock_staff_functions	072 platform_auto_accept
005 allergens_and_orders	039 rls_lock_new_tables	073 invoice_pipeline
006 order_status	040 bill_no_lazy_and_otp_rename	074 tablet_billing_settings
007 order_table_edit	041 floor_state_brain ★	075 billing_info_settings
008 search_alias_and_calls	042 staff_actions_log	076 pending_gate_and_table_session_id
009 order_payment	043 inr_base_currency	077 guest_ban_system
010 dish_options	044 business_day_counter	078 tenancy_core ★ (multi-tenant)
011 table_count	045 dish_ready_state	079 tenant_uniqueness
012 order_archived	046 oplog_device_id	080 tenant_counters
013 realtime_settings	047 blocklist_device	081 staff_rpc_scoping
014 sessions_v2_schema	048 set_member_name	082 tenant_index_scoping
015 sessions_v2_rpcs	049 staff_order_opens_session	083 guest_rpc_scoping_param
016 session_state_bill	050 guard_waiter_calls	084 guest_rpc_derive_scoping
017 call_waiter_guard	051 concurrency_integrity_guards	085 scope_settings_reads
018 head_model_autoopen	052 fk_indexes	086 scope_realtime_breadcrumbs
019 session_shared_cart	053 log_retention	087 restaurant_branding
020 leave_and_close_cleanup	054 staff_users	088 owner_overview
021 waiter_controlled_open	055 staff_users_hardening	089 owner_analytics
022 order_member_attribution	056 staff_users_self_reset	091 roles_and_permissions
023 leave_keeps_table_open	057 realtime_events ★	092 owner_restaurant_link
024 no_orphan_waiter_calls	057 staff_users_self_set_pin	093 grandfather_r1_manager_powers
025 multiple_distinct_calls	058 realtime_orders_triggers	094 issues
026 leave_resolves_calls	059 realtime_ops_triggers	095 orders_analytics_indexes
027 session_state_call_notes	060 realtime_prune_cron	096 targeted_refetch_breadcrumb_gaps ★
028 session_state_orders	061 shift_realtime_breadcrumb	097 multi_owner_join
029 server_authoritative_pricing ★	062 delete_order_item_reconcile	098 perf_indexes
030 real_reviews	063 staff_edit_order	099 auto_close_idle_sessions
031 align_addon_pricing	064 staff_profile_confirmed	100 floor_bundle_rpc ★
032 dish_no	065 session_state_item_details	
033 session_state_removed	066 realtime_content_triggers	
034 close_vs_decline_and_join_race	068 fix_order_items_json_sync / order_item_edit_markers	
101 table_view_summary_rpc ★	102 open_all_tables_rpc	103 close_all_tables_rpc

★ = especially load-bearing: 029 pricing, 041 floor brain, 057 breadcrumb table, 078 multi-tenant core, 096 breadcrumb gap fixes, 100 single-round-trip floor read, 101 slim Table-view summary (300-table scaling).

Where a new developer should start

1. **Read the rules:** `CLAUDE.md` at the repo root is a dense, current operating manual (conventions, gotchas, the egress playbook).
 2. **Trace one order:** follow Section 8 in the code — `public/panels/editor/app.js` (`startOrderWatch` → `pollTables` → `reconcileBoard`), `public/panels/realtime.js`, the `lfh_rt_emit` trigger (migrations 057–066, 086, 096), and `app/api/editor/[...path]/route.ts`.
 3. **Understand the money path:** `lib/money.mjs` + migration 029 + `npm run test:totals`.
 4. **See the floor "brain":** migration 041 (`lfh_floor_state`) and 100 (`lfh_floor_bundle`).
 5. **Reproduce the pain:** run `scripts/stress-max.mjs` against the sandbox and profile `/manager` (Problem 1).
-

End of handoff. Questions about any section map directly to the file paths cited — every claim here is traceable to code in the repo.